

AI STUDY SESSIONS • SESSION 4

From Counting to Learning

The same bigram model, rebuilt as a neural network — plus a live agent demo

makemore, part 1 — second half

How the Hour Runs



08 min

Recap

Where we left the bigram model, and what micrograd gave us



32 min

The neural net reframe

Training set → one-hot → logits → softmax → loss → gradient descent



12 min

Live demo

An agent playing a browser game, driven from Discord



08 min

Wrap & questions

What carries forward into Session 5

Where We Are in the Series

1

Foundation models

Chip Huyen, Ch. 1

2

Counting bigram

makemore — first half

3

micrograd

backprop from scratch

4

Neural bigram

makemore — second half

5

MLP & embeddings

makemore part 2



Why Session 3 was a detour

We stopped Session 2 halfway through the video on purpose. The second half rebuilds the model as a neural network and trains it with gradient descent — which is unreadable if `loss.backward()` is still magic. `micrograd` removed the magic. Today we spend it.

Recap: The Model We Already Built



The dataset

32,033 names, split into character pairs. Special '.' token marks start and end.



Training = counting

Walk every name, tally each (previous char → next char) pair into a 27x27 grid N.



Counts → probabilities

Normalise each row so it sums to 1. That row is a probability distribution.



Sampling

Start at '.', draw the next character from its row, repeat until '.' comes back.

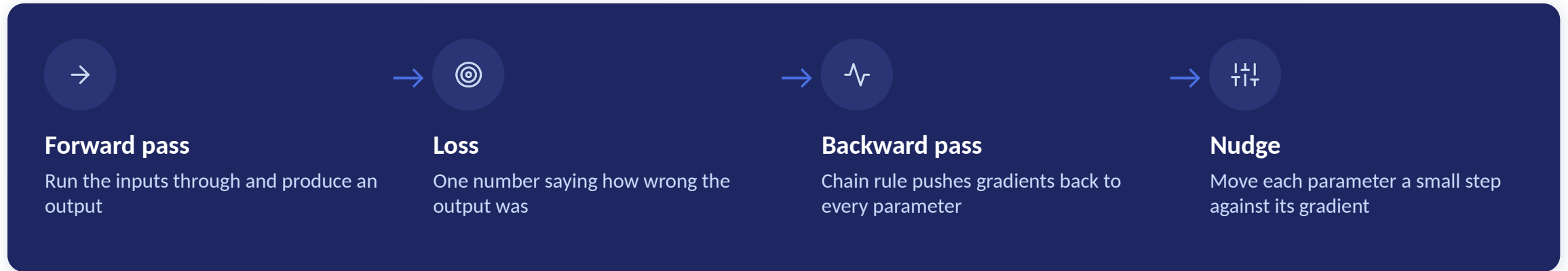
THE SCORE TO BEAT

2.454

Average negative log likelihood of the counting model on the training set.

Remember this number. It is the target the neural network has to match today.

Recap: What micrograd Gave Us



The one sentence to carry into today

Gradient descent is a general-purpose way to tune any set of numbers so that a loss goes down. It does not care whether those numbers are the weights of a tiny neuron or a 27x27 table of character statistics — and that is exactly the door we walk through today.

Same Job, Different Machinery

Both models answer one question: given this character, what comes next? Only the way they arrive at the answer changes.

COUNTING APPROACH

How it learns

Tally every pair once, in a single pass

What it stores

A 27x27 grid of counts

Getting better

There is no 'better' — the counts are what they are

Scaling context

Table size explodes: 27 → 729 → 19,683 → 14.3M rows

GRADIENT APPROACH

How it learns

Guess, measure the loss, nudge the weights, repeat

What it stores

A 27x27 matrix of weights

Getting better

Every iteration lowers the loss a little

Scaling context

Swap the architecture, keep the training loop

The punchline: both land on almost exactly the same loss. The second one is the only one with a future.

Step 1 — Build an Explicit Training Set

Counting never needed labelled examples. Gradient descent does — every bigram becomes an (input, target) pair.

```
xs, ys = [], []
for w in words:
    chs = ['.'] + list(w) + ['.']
    for ch1, ch2 in zip(chs, chs[1:]):
        xs.append(stoi[ch1])
        ys.append(stoi[ch2])
xs = torch.tensor(xs); ys = torch.tensor(ys)
```

For the name "emma"

. → e
e → m
m → m
m → a
a → .

228,146

training examples

27

possible inputs

27

possible targets

Step 2 — One-Hot Encoding

A network multiplies and adds. Feeding it the integer 5 would imply 5 is 'bigger' than 2 — but 'f' is not bigger than 'c'.

The fix

Replace each integer with a row of 27 numbers: a single 1 in that character's slot, zeros everywhere else. No ordering is implied, because every character sits the same distance from every other.

The row is 27 long because the vocabulary is 26 letters plus the '.' token.

'c' → index 3



```
xenc = F.one_hot(xs, num_classes=27).float()
```

The .float() matters — one_hot returns integers, and matrix multiplication needs floats.



The trick worth noticing

Multiplying a one-hot row by a matrix just selects one row of that matrix. The multiplication is doing lookup. Hold that thought — in Session 5 we drop the pretence and call it an embedding table.

Step 3 — The Entire Network

No hidden layers. No bias. No activation function. One matrix.

```
W = torch.randn((27, 27), requires_grad=True)
logits = xenc @ W
```

729

parameters in the whole model



Why 27 x 27

27 inputs in, 27 scores out — one score for every character that could come next.



requires_grad=True

Tells PyTorch to build the graph and fill in W.grad when we call backward().



What logits are

Raw, unbounded scores. They can be negative. They do not sum to anything. Softmax fixes that next.

Step 4 — Softmax: Scores into Probabilities

Two operations, and both of them are things we already did by hand in Session 2.

logits

Raw scores from the matrix. Any sign, any size.



exp()

Everything becomes positive. Read these as counts.



normalise

Divide each row by its sum. Now it is a distribution.

```
counts = logits.exp()  
probs  = counts / counts.sum(1, keepdims=True)
```



So the network's weights are learning log-counts. `exp()` undoes the log, and normalising is the exact same step we did to the tally grid — softmax is not a new idea, it is Session 2's arithmetic with a name.

Five Lines Is the Whole Model



```
xenc = F.one_hot(xs, num_classes=27).float()
logits = xenc @ W
counts = logits.exp()
probs = counts / counts.sum(1, keepdims=True)
loss = -probs[torch.arange(len(xs)), ys].log().mean()
```

Everything from here on is the training loop — the model itself does not change again this session.

Step 5 — The Loss

Exactly the negative log likelihood from Session 2 — nothing new, but now it is differentiable.



Pluck

For every example, pull out the probability the model gave to the character that actually came next.



log

Turns a probability into a score from $-\infty$ (terrible) up to 0 (perfect certainty).



negate

Flip the sign so lower is better, and we have something to minimise.



mean

Average across all 228,146 examples into a single number.

WHY NEGATIVE LOG

probability	-log
1.0	0.00
0.5	0.69
0.1	2.30
0.01	4.61

*Confidence in the right answer costs nothing.
Confidence in the wrong one is punished hard.*

Step 6 — The Training Loop

This loop is the whole of deep learning. Every model in the rest of the series uses this same four-step rhythm.

```
for k in range(200):  
    # forward  
    logits = xenc @ W  
    counts = logits.exp()  
    probs = counts / counts.sum(1, keepdims=True)  
    loss = -probs[torch.arange(num), ys].log().mean()  
    # backward  
    W.grad = None  
    loss.backward()  
    # update  
    W.data += -50 * W.grad
```



Forward

Compute predictions and the loss



Reset the gradient

W.grad = None — PyTorch accumulates, so stale gradients add up



Backward

Fills W.grad with the slope for all 729 weights



Step

Move against the gradient; 50 is the learning rate



Forgetting W.grad = None is the classic bug. The gradients silently pile up, the steps get wilder every iteration, and the loss climbs instead of falling. Everyone writes this bug once.

RESULT

Two Roads, One Destination

After a couple of hundred iterations the network arrives at essentially the counting model's answer.

COUNTING MODEL

2.454

One pass. Tally, normalise, done.

NEURAL MODEL

~2.45

200 iterations of gradient descent.



The weights converge to the log-counts

Gradient descent rediscovers the tally grid — it just gets there by trial and error rather than by looking.



Sampling is unchanged

Same code, same quality of output. The names are still mostly unpronounceable, because a bigram only ever sees one character back.

Smoothing, Reinvented as Regularization

In Session 2 we added a fake count to every cell so nothing had zero probability. The neural version is one extra term in the loss.

```
loss = -probs[torch.arange(num), ys].log().mean() + 0.01 * (W**2).mean()
```

#

Session 2: add fake counts

$N + 1$ pushed every cell away from zero, so no bigram was ever assigned an impossible probability.

⇒

Session 4: penalise big weights

Adding the mean of W squared makes large weights expensive, so the optimiser keeps W close to uniform unless the data really insists.

|||

The strength is a dial

0 means no smoothing at all. Turn it up and every prediction drifts toward 'any letter is equally likely'.

So Why Bother, If the Loss Is the Same?

Because the counting model is a dead end and the neural one is a starting point.



It takes any input

Two characters back? Ten? A whole paragraph? The training loop does not change — only the shape of the input does.



It does not explode

A three-character counting table needs 19,683 rows and a five-character one needs 14.3 million, against a dataset of only 228k pairs.



The loop is universal

Forward, loss, backward, step. Every model for the rest of this series — MLP, WaveNet, transformer — is this loop with a different middle.

Today we did not build a better model. We built a model that can be made better.

Vocabulary You Now Own

Every one of these was something you did today before it was a word you knew.

one-hot

A character as a row of 27 numbers with a single 1

logits

Raw unbounded scores, before softmax

softmax

exp, then normalise — scores become probabilities

NLL

The loss: average of $-\log(\text{probability of the true next character})$

regularization

A loss term that expresses a preference about the weights

learning rate

How far to move on each step; 50 here, and unusually large

PART TWO

An Agent That Plays a Game

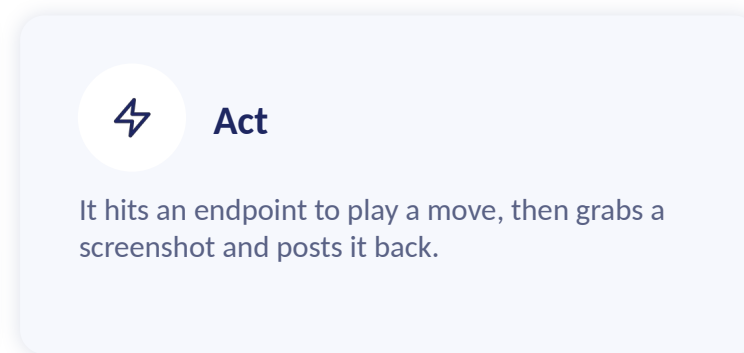
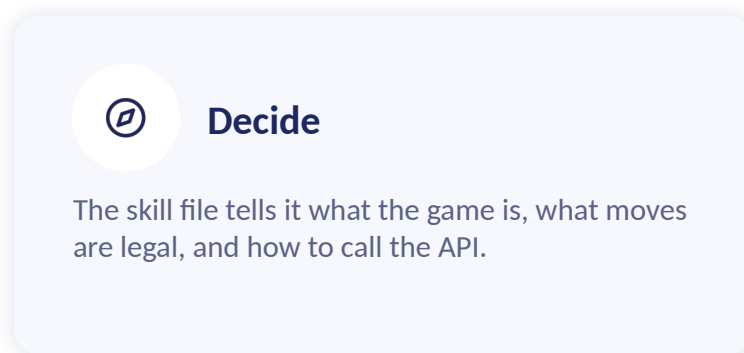
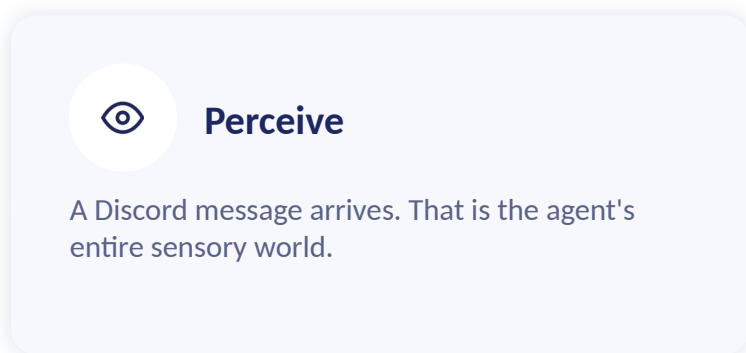
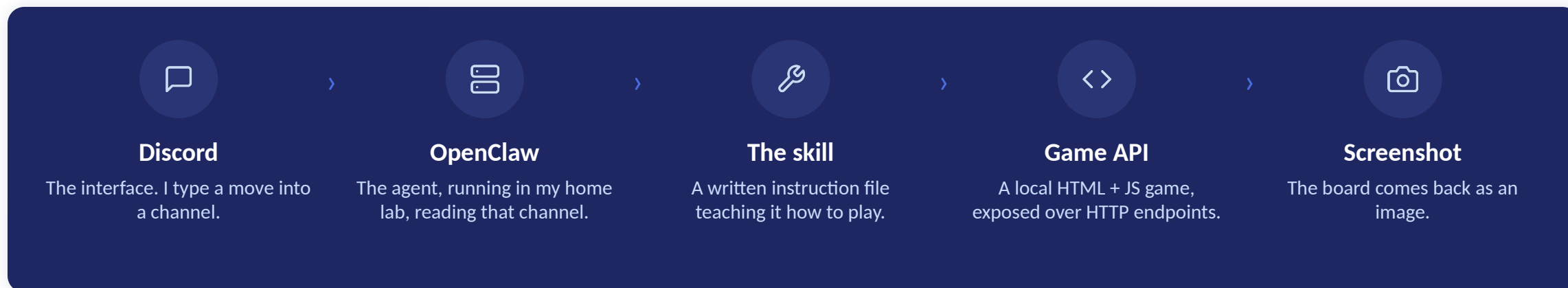
Everything above was models. This is the other half of the job: the system you build around one.



LIVE DEMO

What I Built

A self-hosted coding agent, reachable from Discord, playing a browser game that runs on my own machine.



Where It Broke: The Context Window Wall

The interesting part of this build was not the game. It was discovering that most models could not run the agent at all.

ATTEMPT 1

Local model on my own machine

Free and private, but too slow, and the context window could not hold the agent's instructions plus the skill plus the running history.

ATTEMPT 2

A free hosted tier

Faster, but the same wall from a different direction — the agent's own scaffolding filled the budget before the task even started.

ATTEMPT 3

A paid subscription model

Worked immediately. Not because it was smarter, but because there was finally room for everything the agent needed to hold at once.

The lesson: agent frameworks are context-hungry before they do anything useful. A small model is not a cheaper big model — it is a smaller room.

Three Things That Transfer



Agents are a loop, not a model

Perceive, decide, act, observe the result, repeat. Swapping the model behind it changes the quality of the decisions, not the shape of the system.



Skills are just written instructions

Teaching the agent to play the game meant writing down the rules in plain language. No fine-tuning, no training — the capability lives in text.



Context is the real constraint

Not intelligence, not speed. How much the agent can hold in mind at once decides whether the whole thing works or falls over.



This is the layer you will actually work in — the model is a component, and the system around it is the job.

BEFORE NEXT TIME

Homework & What's Coming



Retrain with a different learning rate

Try 10 and try 200. Watch where it stalls and where it blows up.



Sweep the regularization strength

Set it to 0, then to 1. Sample names at each and describe what changed.



Extend to a trigram

Two characters of context. The training loop should barely change — that is the point.

SESSION 5

MLP & Embeddings

makemore part 2. More context, a hidden layer, and the moment one-hot lookup gets renamed an embedding table.

Questions?